DATA QUALITY ENGINEERING

Data Quality
Test Plan

Aligned to Database Coding & Naming Standards | Enterprise DG Framework v2.0
Platform Snowflake + Databricks / Delta Live Tables
Tooling dbt · Great Expectations · Snowflake Tasks · Databricks DLT
Dataset Scope fact_order · dim_customer · fact_resource_allocation · stg_order · raw_order_event
DQ Rules In Scope DQR-001 through DQR-010 + 10 new rules defined in this plan
Document Owner Data Quality Lead · Reviewed: Data Steward · Approved: Data Owner

1. Purpose & Scope

This Test Plan defines the complete validation strategy for the data platform described in the Enterprise Data Governance & Quality Transformation case study. It operationalises the DQ Framework, Rules Catalog (DQR-001 through DQR-010), and enforcement standards defined in the Database Coding & Naming Standards document (v2.0) into executable, traceable test cases across every layer of the Medallion architecture.

The plan covers five areas of validation: structural and schema compliance, data quality dimension rules, cross-platform consistency, audit column integrity, and pipeline SLA monitoring. Every test case maps to a DQ rule, a governance framework section, a dataset tier, and a specific enforcement tool. Nothing in this plan is advisory — each test either passes or triggers a defined action.

What This Plan Governs
‣ Schema and naming standard compliance — enforced at CI/CD (sqlfluff + dbt compile)
‣ Data quality dimension tests — Completeness, Uniqueness, Validity, Consistency, Timeliness, Referential Integrity
‣ DQ Rules Catalog DQR-001 through DQR-010 — full test case implementation for each rule
‣ 10 additional rules extending the catalog (DQR-011 through DQR-020)
‣ Audit column integrity — mandatory columns per Section 4.7 of the framework ‣ Cross-platform reconciliation — Snowflake vs. Databricks fact layer alignment ‣ Pipeline SLA monitoring — freshness and volume anomaly detection per tier
‣ Simulated data scenarios — positive, negative, and boundary test cases per rule

1.1 Datasets in Scope

Dataset	Schema / Layer	Tier	Primary Consumers	DQ Rules Applied
fact_order	marts / Gold	**Critical**	BI dashboards, Finance reporting, ML features	DQR-001 to 003, 006, 007, 009 + DQR-011 to 015
dim_customer	marts / Gold	**Critical**	All BI dashboards, fact_order FK ref	DQR-005, 008, 010 + DQR-016 to 018
stg_order	staging / Silver	**Tier 2**	fact_order upstream	DQR-001, 002 + DQR-019
raw_order_event	raw / Bronze	**Tier 2**	stg_order upstream	Schema check, audit cols, volume

fact_resource_allocation	marts / Gold	**High**	Ops dashboard, capacity planning	DQR-004 + DQR-020

Tier 1 datasets (fact_order, dim_customer) require DQS ≥ 95, real-time anomaly detection, and a 1-hour freshness SLA per framework Section 7.2. All test failures on Tier 1 assets trigger the Critical severity response: pipeline halt + PagerDuty + Slack #data-incidents.

1.2 Test Layers & Enforcement Mapping

Test Layer	When It Runs	Tool	Failure Behaviour	Tier Applicability
CI/CD — Naming & Schema	Every pull request	sqlfluff + dbt compile	PR blocked — cannot merge	All tiers
dbt — Model Promotion Gate	Every dbt run (CI + scheduled)	dbt test	Model not promoted to next layer	Tier 1 & 2
Great Expectations — Post-Load	After each curated layer refresh	GE checkpoint	Pipeline halted; incident raised (Tier 1)	Tier 1 required; Tier 2 recommended
Snowflake Tasks — Continuous	Every 30 min (Tier 1), 1 hr (Tier 2)	Snowflake Alert / Task	Slack alert → Custodian; escalation on breach	Tier 1 & 2
Databricks DLT — Stream Gate	On each DLT pipeline run	@dlt.expect_or_fail/drop	Row dropped or pipeline halted per severity	Tier 1 (expect_or_fail); Tier 2 (expect_or_drop)
Manual / Audit	Quarterly catalog review	SQL + governance review	Remediation ticket; exception process	All tiers

2. Test Environment & Data Strategy

2.1 Environment Architecture

Tests execute across three environments. Each environment has independent Snowflake databases and Databricks workspaces. No test data from DEV or QA can propagate to PROD. The naming convention enforces this structurally — the database prefix (dev_, qa_) is validated by sqlfluff at CI.

Environment	Snowflake DB	Databricks Catalog	Purpose	Data Source
DEV	commerce_dev	commerce_dev	Unit tests, rule development, schema iteration	Synthetic generated data (see Section 2.2)
QA / STAGING	commerce_staging	commerce_staging	Integration tests, regression suite, GE checkpoint dry-runs	Anonymised production subset (30-day window)
PROD	commerce_prod	commerce_prod	Live monitoring, SLA checks, DQS calculation	Live operational data

2.2 Data Strategy — Simulated Test Data

You do not need to install MySQL or MSSQL for this test plan. All test execution happens natively in Snowflake and Databricks, which are already the production platforms. The only requirement is a populated DEV environment. Two approaches are available:

Option A — Synthetic Data via SQL (Recommended, No External Tools)

Insert controlled datasets directly into the DEV schema using Snowflake's generator functions. This approach requires no additional software and produces deterministic data for positive, negative, and boundary tests.

```
-- Create DEV test schema (compliant naming: dev_ prefix per framework Section 4.2)
CREATE SCHEMA IF NOT EXISTS commerce_dev.staging;
CREATE SCHEMA IF NOT EXISTS commerce_dev.marts;

-- Seed: fact_order base dataset (1,000 clean records)
CREATE OR REPLACE TABLE commerce_dev.marts.fact_order AS
SELECT
    UUID_STRING()                                      AS order_id,
    UUID_STRING()                                      AS customer_id,
    DATEADD('day', -MOD(SEQ4(), 90), CURRENT_DATE())  AS order_date,
    ROUND(UNIFORM(10.0, 5000.0, RANDOM()), 4)          AS order_amount,
    CASE MOD(SEQ4(), 5)
        WHEN 0 THEN 'pending'
        WHEN 1 THEN 'confirmed'
        WHEN 2 THEN 'shipped'
        WHEN 3 THEN 'delivered'
```

```
        ELSE 'cancelled'
    END                                                 AS order_status_code,
    CURRENT_TIMESTAMP()                                 AS audit_insertion_timestamp,
    CURRENT_TIMESTAMP()                                 AS audit_updated_timestamp,
    UUID_STRING()                                       AS audit_batch_id,
    'ERP_SAP'                                           AS source_system_id,
    SHA2(UUID_STRING(), 256)                            AS record_hash
FROM TABLE(GENERATOR(ROWCOUNT => 1000));

-- Seed: dim_customer (500 clean records)
CREATE OR REPLACE TABLE commerce_dev.marts.dim_customer AS
SELECT
    UUID_STRING()                                       AS customer_id,
    SEQ4() + 1                                          AS customer_sk,
    'Customer_' || SEQ4()                               AS customer_name,
    CASE MOD(SEQ4(), 4)
        WHEN 0 THEN 'ENTERPRISE'
        WHEN 1 THEN 'SMB'
        WHEN 2 THEN 'CONSUMER'
        ELSE 'PARTNER'
    END                                                 AS customer_segment_code,
    CASE MOD(SEQ4(), 3)
        WHEN 0 THEN 'US' WHEN 1 THEN 'GB' ELSE 'DE'
    END                                                 AS country_code,
    TRUE                                                AS is_active_flag,
    TRUE                                                AS is_current_flag,
    CURRENT_TIMESTAMP()                                 AS audit_insertion_timestamp,
    NULL::TIMESTAMP_TZ                                  AS audit_updated_timestamp,
    UUID_STRING()                                       AS audit_batch_id,
    'CRM_SALESFORCE'                                    AS source_system_id,
    SHA2(UUID_STRING(), 256)                            AS record_hash
FROM TABLE(GENERATOR(ROWCOUNT => 500));
```

Option B — Anonymised Production Subset (QA Environment)

For integration and regression testing, a 30-day anonymised extract of production data is loaded into the QA environment. PII columns (customer_name, email) are masked via Snowflake column masking policies per framework Section 11.2 before export. This approach tests against real-world data distributions but contains no identifiable personal data.

Data Strategy Decision Summary
▸ DEV — 100% synthetic, generated via SQL GENERATOR function. Zero external tools required.
▸ QA — Anonymised production subset. PII masked before transfer. Validates against real distributions.
▸ PROD — Live data. Read-only for monitoring and DQS calculation. No test writes to PROD.
▸ MySQL / MSSQL — NOT required. All tests run natively on Snowflake and Databricks.
▸ If simulating ingestion from an external source, use Snowflake Stage + COPY INTO from local CSV files.

2.3 Negative Test Data Injection

For each test case, a controlled injection script inserts specifically malformed records to verify that the validation catches them. These scripts run only in DEV and are version-controlled in the test suite repository. Each injection is paired with a cleanup script to restore the baseline state.

```
-- Negative data injection: DQR-001 (NULL primary key)
INSERT INTO commerce_dev.marts.fact_order
    (order_id, customer_id, order_date, order_amount,
     order_status_code, audit_insertion_timestamp, audit_batch_id,
     source_system_id, record_hash)
VALUES
    (NULL, UUID_STRING(), CURRENT_DATE(), 99.99,
     'confirmed', CURRENT_TIMESTAMP(), 'TEST_BATCH_001',
     'TEST', SHA2('null_pk_test', 256));

-- Negative data injection: DQR-002 (duplicate primary key)
INSERT INTO commerce_dev.marts.fact_order
SELECT order_id, customer_id, order_date, order_amount,
       order_status_code, CURRENT_TIMESTAMP(), audit_updated_timestamp,
       'TEST_BATCH_DUP', source_system_id, record_hash
FROM   commerce_dev.marts.fact_order
LIMIT  5;   -- duplicates the first 5 order_ids

-- Negative data injection: DQR-006 (negative order_amount)
INSERT INTO commerce_dev.marts.fact_order
    (order_id, customer_id, order_date, order_amount,
     order_status_code, audit_insertion_timestamp, audit_batch_id,
     source_system_id, record_hash)
VALUES
    (UUID_STRING(), UUID_STRING(), CURRENT_DATE(), -250.00,
     'confirmed', CURRENT_TIMESTAMP(), 'TEST_BATCH_NEG',
     'TEST', SHA2('neg_amount_test', 256));

-- Cleanup: remove all test-injected records
DELETE FROM commerce_dev.marts.fact_order
WHERE source_system_id = 'TEST';
```

3. Test Cases — DQ Rules Catalog (DQR-001 to DQR-010)

Each test case below maps directly to an active rule in the DQ Rules Catalog. Test cases include the validation query, expected result, enforcement tool, injection script reference, and pass/fail criteria. All Critical rules must achieve 100% pass rate before any dataset is promoted from staging to curated.

TC-001 | Primary Key Not Null — DQR-001 | Critical

Field	Value
Rule Reference	DQR-001
Target Table	fact_order
DQ Dimension	Completeness
Tier	1 — Critical (pipeline halt on failure)
Framework Section	7.3 — Data Health Rules | 5 — Data Type Alignment
Enforcement Tools	dbt: not_null test | GE: expect_column_values_to_not_be_null | Snowflake Task
Test Type	Positive (clean data passes) + Negative (NULL injection fails)

```
-- TC-001: Validation query (runs via dbt test + Snowflake Task)
SELECT COUNT(*) AS null_pk_count
FROM   commerce_prod.marts.fact_order
WHERE  order_id IS NULL;

-- PASS criteria:  null_pk_count = 0
-- FAIL criteria:  null_pk_count > 0  →  halt pipeline, page Steward

-- dbt schema.yml equivalent:
-- columns:
--   - name: order_id
--     tests: [ not_null ]

-- GE equivalent:
-- validator.expect_column_values_to_not_be_null('order_id')
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: clean baseline	1,000 records, all order_id populated	null_pk_count = 0	**PASS**
Negative: single NULL PK	1 record with order_id = NULL injected	null_pk_count = 1	**FAIL — HALT**
Negative: bulk NULL PKs	50 records with order_id = NULL injected	null_pk_count = 50	**FAIL — HALT**

Boundary: NULL + valid mix	999 valid + 1 NULL	null_pk_count = 1	**FAIL — HALT**

TC-002 | Primary Key Uniqueness — DQR-002 | Critical

Field	Value
Rule Reference	DQR-002
Target Table	fact_order
DQ Dimension	Uniqueness
Tier	1 — Critical (pipeline halt on failure; P1 ticket auto-created)
Framework Section	7.3 — Data Health Rules | 10.1 — dbt mandatory unique test
Enforcement Tools	dbt: unique test | GE: expect_column_values_to_be_unique | Snowflake Task
Test Type	Positive + Negative + Boundary

```
-- TC-002: Duplicate PK detection
SELECT   order_id,
         COUNT(*) AS dup_count
FROM     commerce_prod.marts.fact_order
GROUP BY order_id
HAVING   COUNT(*) > 1
ORDER BY dup_count DESC;

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows returned  →  halt pipeline, open P1 ticket

-- Extended: percentage impact assessment
WITH dups AS (
    SELECT order_id, COUNT(*) AS cnt
    FROM   commerce_prod.marts.fact_order
    GROUP BY order_id HAVING COUNT(*) > 1
)
SELECT
    COUNT(*)                                    AS duplicate_order_ids,
    SUM(cnt - 1)                                AS excess_records,
    ROUND(SUM(cnt-1)*100.0 / (SELECT COUNT(*)
          FROM commerce_prod.marts.fact_order), 4) AS pct_of_total
FROM dups;
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: clean baseline	1,000 records, all order_id unique	0 rows returned	**PASS**
Negative: 5 exact duplicates	5 order_ids duplicated (10 excess records)	5 rows returned	**FAIL — HALT**
Negative: 1 duplicate	1 order_id appears twice	1 row returned	**FAIL — HALT**

Boundary: near-duplicate keys	Two UUIDs differing by 1 character	0 rows (correctly)	**PASS**

TC-003 | FK Integrity: fact_order → dim_customer — DQR-003 | Critical

Field	Value
Rule Reference	DQR-003
Target Table	fact_order (FK: customer_id → dim_customer.customer_id)
DQ Dimension	Referential Integrity
Tier	1 — Critical (alert Steward; block downstream marts)
Framework Section	7.3 — FK: 100% referential integrity required | 10.1 — dbt relationships test
Enforcement Tools	dbt: relationships test | GE: custom SQL expectation | Snowflake Task
Test Type	Positive + Negative (orphan injection) + Cascade test

```
-- TC-003: Orphan FK detection
SELECT  o.order_id,
        o.customer_id,
        o.order_date
FROM    commerce_prod.marts.fact_order  AS o
LEFT JOIN commerce_prod.marts.dim_customer AS c
       ON o.customer_id = c.customer_id
WHERE   c.customer_id IS NULL;

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows returned  →  alert Steward, block mart refresh

-- Quantified impact version (for DQS calculation):
SELECT
    COUNT(*) AS orphan_count,
    ROUND(COUNT(*) * 100.0 / NULLIF(
          (SELECT COUNT(*) FROM commerce_prod.marts.fact_order), 0), 4
    )        AS orphan_pct
FROM fact_order  AS o
LEFT JOIN dim_customer AS c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: all FKs resolve	1,000 orders, all customer_id present in dim_customer	0 orphan rows	**PASS**
Negative: 10 orphan customer_ids	10 orders with customer_id not in dim_customer	10 orphan rows	**FAIL — ALERT**
Negative: customer deleted from dim	dim_customer row deleted while fact row exists	>= 1 orphan row	**FAIL — ALERT**

Boundary: NULL customer_id	order with customer_id = NULL (also caught by TC-001)	NULL not a valid FK	**FAIL — HALT**

TC-004 | Allocation Percentage Sum ≤ 100% — DQR-004 | High

Field	Value
Rule Reference	DQR-004
Target Table	fact_resource_allocation
DQ Dimension	Validity + Business Rule Conformance
Tier	2 — High (quarantine affected rows; alert Data Owner)
Framework Section	5 — Data Type Alignment: _pct stored as decimal 0–1 | 8 — DQ Rules Catalog
Enforcement Tools	dbt: custom SQL test | GE: expect_column_sum grouped assertion
Test Type	Positive + Negative (sum > 1.0) + Boundary (sum = 1.0 exactly)

```
-- TC-004: Allocation sum validation (framework stores pct as decimal: 0.15 = 15%)
SELECT  resource_id,
        ROUND(SUM(allocation_pct), 4)   AS total_allocation,
        COUNT(*)                         AS row_count
FROM    commerce_prod.marts.fact_resource_allocation
GROUP BY resource_id
HAVING  SUM(allocation_pct) > 1.0000;

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows  →  quarantine resource_ids; alert Data Owner

-- Boundary test: exactly 1.0 must PASS
SELECT resource_id, ROUND(SUM(allocation_pct), 4) AS total
FROM   commerce_prod.marts.fact_resource_allocation
GROUP BY resource_id
HAVING ROUND(SUM(allocation_pct), 4) = 1.0000;
-- Expected: all rows in this result are VALID (not violations)
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: sums exactly 1.0	resource_id RES-001: 3 rows summing to 1.0000	0 violation rows	**PASS**
Positive: sums < 1.0 (partial)	resource_id RES-002: 2 rows summing to 0.6000	0 violation rows	**PASS**
Negative: sum = 1.0001	resource_id RES-003: 3 rows summing to 1.0001	1 violation row	**FAIL — QUARANTINE**
Negative: sum = 1.5	resource_id RES-004: over-allocated by 50%	1 violation row	**FAIL — QUARANTINE**

Boundary: sum = 1.0000 exactly	4 decimal place precision: 0.2500 × 4 = 1.0000	0 violation rows	**PASS**

TC-005 | Customer Name Completeness — DQR-005 | High

Field	Value
Rule Reference	DQR-005
Target Table	dim_customer
DQ Dimension	Completeness
Tier	1 — High (alert Steward; log DQ incident)
Framework Section	7.1 — Completeness: % non-null in mandatory fields | 4.4 — _name: VARCHAR, non-null
Enforcement Tools	dbt: not_null test | GE: expect_column_values_to_not_be_null
Test Type	Positive + Negative + Bulk null scenario

```
-- TC-005: Customer name completeness
SELECT  COUNT(*)                          AS total_customers,
        SUM(CASE WHEN customer_name IS NULL THEN 1 ELSE 0 END)
                                          AS null_name_count,
        ROUND(
            SUM(CASE WHEN customer_name IS NULL THEN 1 ELSE 0 END)
            * 100.0 / NULLIF(COUNT(*), 0), 4
        )                                 AS null_name_pct
FROM    commerce_prod.marts.dim_customer;

-- PASS criteria:  null_name_count = 0
-- FAIL criteria:  null_name_count > 0  →  alert Steward, log to DQ incident table

-- Tier 1 threshold check (Completeness DQS must be >= 99.5%):
-- If null_name_pct > 0.5, DQS Completeness component drops below threshold
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: all names present	500 customers, all customer_name populated	null_name_count = 0	**PASS**
Negative: 1 NULL name	1 customer with customer_name = NULL	null_name_count = 1	**FAIL — ALERT**
Negative: empty string name	1 customer with customer_name = ''	Caught by TC-017	**FAIL — ALERT**
Boundary: whitespace-only name	customer_name = ' ' (3 spaces)	Caught by TC-017	**FAIL — ALERT**

TC-006 | Order Amount Non-Negative — DQR-006 | High

Field	Value
Rule Reference	DQR-006
Target Table	fact_order
DQ Dimension	Validity
Tier	1 — High (quarantine negative rows; alert Steward)
Framework Section	5 — _amount: NUMBER(18,4); CHECK (value >= 0) required | 8 — Rules Catalog
Enforcement Tools	dbt: expression_is_true | GE: expect_column_values_to_be_between
Test Type	Positive + Negative (zero, negative) + Boundary

```
-- TC-006: Amount validity check
SELECT  order_id,
        order_amount,
        order_status_code
FROM    commerce_prod.marts.fact_order
WHERE   order_amount < 0
   OR   order_amount IS NULL;

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows  →  quarantine; alert Steward

-- Note: order_amount = 0 is a separate business rule (DQR-011)
-- This rule focuses on strictly negative values only.
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: all amounts > 0	1,000 orders, amount BETWEEN 0.01 AND 5000.00	0 violation rows	**PASS**
Negative: amount = -1.00	1 order with order_amount = -1.00	1 violation row	**FAIL — QUARANTINE**
Negative: amount = -0.0001	Sub-cent negative value	1 violation row	**FAIL — QUARANTINE**
Boundary: amount = 0.00	order_amount exactly zero	Caught by DQR-011	**FAIL — ALERT**
Boundary: amount = 0.0001	Smallest valid positive value	0 violations	**PASS**

TC-007 | Order Date Not Future — DQR-007 | Medium

Field	Value
Rule Reference	DQR-007

Target Table	fact_order
DQ Dimension	Validity
Tier	1 — Medium (log warning; flag affected records)
Framework Section	5 — _date: DATE type; valid date range via CHECK constraint | 8 — Rules Catalog
Enforcement Tools	dbt: expression_is_true | GE: expect_column_max_to_be_between
Test Type	Positive + Negative (future date) + Boundary (today = valid)

```
-- TC-007: Future order date detection
SELECT  order_id,
        order_date,
        DATEDIFF('day', CURRENT_DATE(), order_date) AS days_in_future
FROM    commerce_prod.marts.fact_order
WHERE   order_date > CURRENT_DATE();

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows  →  log warning; add is_flagged_flag = TRUE

-- GE equivalent:
-- validator.expect_column_max_to_be_between(
--     column='order_date', max_value=datetime.date.today()
-- )
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: all dates historical	All order_date <= CURRENT_DATE()	0 rows	**PASS**
Positive: order_date = today	order_date = CURRENT_DATE() exactly	0 rows	**PASS**
Negative: order_date = tomorrow	order_date = CURRENT_DATE() + 1	1 violation	**FAIL — LOG**
Negative: order_date = +1 year	order_date = DATEADD('year', 1, TODAY())	1 violation	**FAIL — LOG**

TC-008 | Boolean Not Null: is_active_flag — DQR-008 | Medium

Field	Value
Rule Reference	DQR-008
Target Table	dim_customer
DQ Dimension	Completeness + Schema Compliance
Tier	1 — Medium (log warning; apply DEFAULT value TRUE)

Framework Section	4.4 — _flag: BOOLEAN, NOT NULL, DEFAULT required | 5 — Boolean data type constraint
Enforcement Tools	dbt: not_null + accepted_values | GE: expect_column_values_to_not_be_null
Test Type	Positive + Negative (NULL) + Negative (non-boolean value)

```
-- TC-008: Boolean column completeness
SELECT  COUNT(*) AS null_flag_count
FROM    commerce_prod.marts.dim_customer
WHERE   is_active_flag IS NULL;

-- PASS criteria:  null_flag_count = 0
-- FAIL criteria:  any NULLs  →  apply DEFAULT TRUE; log warning

-- dbt schema.yml equivalent:
-- - name: is_active_flag
--   tests:
--     - not_null
--     - accepted_values:
--         values: [true, false]
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: all flags TRUE or FALSE	500 customers, is_active_flag populated	null_flag_count = 0	**PASS**
Negative: 1 NULL flag	1 customer with is_active_flag = NULL	null_flag_count = 1	**FAIL — DEFAULT**
Negative: negative logic column	Column named is_not_active_flag present	sqlfluff CI blocks	**FAIL — CI BLOCK**

TC-009 | Freshness SLA: fact_order — DQR-009 | Critical

Field	Value
Rule Reference	DQR-009
Target Table	fact_order
DQ Dimension	Timeliness
Tier	1 — Critical (alert Steward; page on-call engineer within 15 min)
Framework Section	7.2 — Tier 1 freshness SLA: ≤ 1 hour | 10.3 — Snowflake Alert pattern
Enforcement Tools	Snowflake Alert (runs every 10 min) | dbt source freshness | Snowflake Task
Test Type	Positive (within SLA) + Negative (breach 1 hr) + Critical (breach 4 hr)

```
-- TC-009: Freshness SLA evaluation (Snowflake Task, every 10 min)
SELECT
```

```
    MAX(audit_insertion_timestamp)                          AS last_loaded_at,
    CURRENT_TIMESTAMP()                                     AS checked_at,
    DATEDIFF('minute',
        MAX(audit_insertion_timestamp),
        CURRENT_TIMESTAMP())                                AS lag_minutes,
    CASE
        WHEN DATEDIFF('minute',
             MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 240  THEN 'CRITICAL_BREACH'
        WHEN DATEDIFF('minute',
             MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 60   THEN 'SLA_BREACH'
        WHEN DATEDIFF('minute',
             MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 45   THEN 'SLA_WARNING'
        ELSE 'WITHIN_SLA'
    END                                                     AS freshness_status
FROM commerce_prod.marts.fact_order;

-- Snowflake Alert implementation:
CREATE OR REPLACE ALERT alert_fact_order_freshness
    WAREHOUSE = monitoring_wh
    SCHEDULE  = '10 MINUTES'
    IF (EXISTS (
        SELECT 1 FROM commerce_prod.marts.fact_order
        HAVING DATEDIFF('minute',
               MAX(audit_insertion_timestamp),
               CURRENT_TIMESTAMP()) > 60
    ))
    THEN CALL SYSTEM$SEND_EMAIL(
        'data-steward@company.com',
        'CRITICAL: fact_order SLA breach',
        'fact_order lag exceeds 60-minute Tier 1 SLA.'
    );
```

Test Scenario	Simulated Condition	Expected Result	Pass / Fail
Positive: lag = 30 min	Last record 30 min ago	WITHIN_SLA	**PASS**
Warning: lag = 50 min	Last record 50 min ago	SLA_WARNING	**WARN**
Breach: lag = 75 min	Last record 75 min ago	SLA_BREACH	**FAIL — ALERT**
Critical: lag = 5 hours	Pipeline stalled 5 hours	CRITICAL_BREACH	**FAIL — PAGE**
Boundary: lag = 60 min	Exactly 60 minutes elapsed	SLA_BREACH	**FAIL — ALERT**

TC-010 | Country Code in Reference List — DQR-010 | Medium

Field	Value
Rule Reference	DQR-010
Target Table	dim_customer
DQ Dimension	Validity + Referential Integrity

Tier	1 — Medium (log warning; route to Steward review)
Framework Section	4.4 — _code: VARCHAR, enum-like | 5 — accepted_values via dbt test
Enforcement Tools	dbt: relationships or accepted_values test | GE: expect_column_values_to_be_in_set
Test Type	Positive (valid ISO codes) + Negative (invalid code) + Boundary (NULL code)

```
-- TC-010: Country code referential validity
SELECT  c.customer_id,
        c.country_code
FROM    commerce_prod.marts.dim_customer AS c
LEFT JOIN commerce_prod.reference.ref_country AS r
       ON c.country_code = r.country_code
WHERE   r.country_code IS NULL;

-- PASS criteria:  0 rows returned
-- FAIL criteria:  any rows  →  log warning; route to Steward

-- GE equivalent for DEV (without ref table):
-- validator.expect_column_values_to_be_in_set(
--     'country_code', ['US','GB','DE','FR','CA','AU','JP', ...]
-- )
```

Test Scenario	Injected Data	Expected Result	Pass / Fail
Positive: valid ISO-3166 codes	US, GB, DE, FR, CA — all in ref_country	0 violation rows	**PASS**
Negative: code = 'XX'	Non-existent country code	1 violation row	**FAIL — LOG**
Negative: code = 'usa' (lower)	Lowercase — should be 'US'	1 violation row	**FAIL — LOG**
Boundary: code = NULL	NULL country_code (also TC-016)	1 violation row	**FAIL — LOG**

4. Extended Test Cases — DQR-011 to DQR-020

The following ten rules extend the Rules Catalog beyond the original DQR-001 to DQR-010. Each rule was identified during the baseline assessment as a gap not covered by the existing catalog. They have been formally authored to the same standard and are ready for Data Steward activation.

TC-011 | Order Amount Zero Check — DQR-011 | Medium

Field	Value
Rule Reference	DQR-011 (new)
Target Table	fact_order
DQ Dimension	Validity
Tier	1 — Medium
Rationale	DQR-006 catches negatives. Zero-amount orders are a distinct business anomaly — a confirmed or delivered order with zero revenue is likely a data defect, not a valid state.
Enforcement Tools	dbt: expression_is_true | GE: expect_column_values_to_be_between

```
-- TC-011: Zero-amount order detection
SELECT  order_id, order_amount, order_status_code
FROM    commerce_prod.marts.fact_order
WHERE   order_amount = 0
  AND   order_status_code IN ('confirmed','shipped','delivered');

-- PASS: 0 rows
-- FAIL: any confirmed/shipped/delivered order with zero revenue → log + Steward
review
```

TC-012 | Order Status Domain Validation — DQR-012 | High

Field	Value
Rule Reference	DQR-012 (new)
Target Table	fact_order
DQ Dimension	Validity
Tier	1 — High (quarantine invalid rows)
Rationale	order_status_code must belong to the approved domain. Values outside this set break downstream BI categorisation, funnel analysis, and revenue-by-status reporting.
Approved Domain	'pending', 'confirmed', 'shipped', 'delivered', 'cancelled'
Enforcement Tools	dbt: accepted_values | GE: expect_column_values_to_be_in_set

```
-- TC-012: Status code domain validation
SELECT  order_id,
        order_status_code,
        audit_insertion_timestamp
FROM    commerce_prod.marts.fact_order
WHERE   order_status_code NOT IN (
            'pending','confirmed','shipped','delivered','cancelled'
        )
   OR   order_status_code IS NULL;

-- PASS: 0 rows
-- FAIL: quarantine affected rows; alert Steward; block mart aggregations
```

TC-013 | Audit Column Completeness — DQR-013 | Critical

Field	Value
Rule Reference	DQR-013 (new)
Target Table	All stg_, int_, dim_, fact_, mart_ tables
DQ Dimension	Completeness + Schema Compliance
Tier	All — Critical (deployment blocker)
Rationale	Framework Section 4.7 mandates audit_insertion_timestamp, audit_batch_id, and source_system_id on all non-raw tables. NULL values in these columns break lineage tracing and pipeline incident investigation.
Enforcement Tools	dbt: not_null tests on all audit columns | sqlfluff: column presence check at CI

```
-- TC-013: Audit column completeness (runs per table)
SELECT
    COUNT(*) AS total,
    SUM(CASE WHEN audit_insertion_timestamp IS NULL THEN 1 ELSE 0 END) AS
null_insertion_ts,
    SUM(CASE WHEN audit_batch_id            IS NULL THEN 1 ELSE 0 END) AS
null_batch_id,
    SUM(CASE WHEN source_system_id          IS NULL THEN 1 ELSE 0 END) AS
null_source_id,
    SUM(CASE WHEN record_hash               IS NULL THEN 1 ELSE 0 END) AS
null_record_hash
FROM commerce_prod.marts.fact_order;

-- PASS: all null counts = 0
-- FAIL: any null count > 0  →  halt pipeline (framework Section 4.7: zero null
tolerance)
```

TC-014 | Record Hash Consistency — DQR-014 | High

Field	Value

Rule Reference	DQR-014 (new)
Target Table	fact_order, dim_customer
DQ Dimension	Consistency + Integrity
Tier	1 & 2 — High
Rationale	record_hash is SHA-256 of business key columns (framework Section 4.7). If two rows have identical hashes, they are identical business events — a deduplication failure. This is a second-layer uniqueness check complementing DQR-002.
Enforcement Tools	dbt: custom SQL test | Snowflake Task

```
-- TC-014: Duplicate record_hash detection
SELECT  record_hash,
        COUNT(*) AS hash_count
FROM    commerce_prod.marts.fact_order
GROUP BY record_hash
HAVING  COUNT(*) > 1;

-- PASS: 0 rows
-- FAIL: quarantine duplicates; open High incident; deduplication re-run triggered
```

TC-015 | Volume Anomaly: fact_order — DQR-015 | High

Field	Value
Rule Reference	DQR-015 (new)
Target Table	fact_order
DQ Dimension	Timeliness + Completeness (via volume signal)
Tier	1 — High
Rationale	A sudden drop or spike in daily order volume is a proxy signal for ingestion failure or source system anomaly. Framework Section 7.2 mandates ±20% volume alerting for Tier 1 datasets.
Enforcement Tools	Snowflake Task (daily) | GE: expect_table_row_count_to_be_between

```
-- TC-015: Volume anomaly detection (framework Section 7.2: alert if ±20% of 7-day
avg)
WITH daily_counts AS (
    SELECT
        order_date,
        COUNT(*) AS daily_record_count
    FROM commerce_prod.marts.fact_order
    WHERE order_date >= DATEADD('day', -8, CURRENT_DATE())
    GROUP BY order_date
),
baseline AS (
    SELECT AVG(daily_record_count) AS avg_7d
    FROM   daily_counts
```

```
    WHERE  order_date < CURRENT_DATE()
)
SELECT
    d.order_date,
    d.daily_record_count,
    ROUND(b.avg_7d, 0)                                    AS baseline_avg,
    ROUND(ABS(d.daily_record_count - b.avg_7d)
          / NULLIF(b.avg_7d, 0) * 100, 2)                AS pct_deviation,
    CASE
        WHEN ABS(d.daily_record_count - b.avg_7d)
             / NULLIF(b.avg_7d, 0) > 0.20
        THEN 'ANOMALY_DETECTED'
        ELSE 'WITHIN_BOUNDS'
    END                                                   AS volume_status
FROM daily_counts d CROSS JOIN baseline b
WHERE d.order_date = CURRENT_DATE();

-- PASS: pct_deviation <= 20%
-- FAIL: pct_deviation > 20%  →  Slack alert; High incident ticket
```

TC-016 | Surrogate Key Sequence Integrity — DQR-016 | High

Field	Value
Rule Reference	DQR-016 (new)
Target Table	dim_customer
DQ Dimension	Uniqueness + Schema Compliance
Tier	1 — High
Rationale	customer_sk is a surrogate key defined as IDENTITY. NULL or non-unique surrogate keys break SCD Type 2 management and corrupt all joins from fact tables using the SK.
Enforcement Tools	dbt: not_null + unique | GE: expect_column_values_to_not_be_null

```
-- TC-016: Surrogate key integrity
SELECT
    SUM(CASE WHEN customer_sk IS NULL THEN 1 ELSE 0 END) AS null_sk_count,
    COUNT(*) - COUNT(DISTINCT customer_sk)               AS duplicate_sk_count
FROM commerce_prod.marts.dim_customer;

-- PASS: both counts = 0
-- FAIL: quarantine affected rows; High alert; SCD logic review triggered
```

TC-017 | Empty String Detection — DQR-017 | Medium

Field	Value
Rule Reference	DQR-017 (new)
Target Table	dim_customer, fact_order

DQ Dimension	Completeness + Validity
Tier	1 & 2 — Medium
Rationale	Empty strings ('') and whitespace-only values pass NOT NULL checks but are semantically equivalent to NULLs. They silently corrupt aggregations, groupings, and lookup joins.
Enforcement Tools	dbt: custom SQL test | GE: expect_column_values_to_match_regex

```
-- TC-017: Empty and whitespace-only string detection
SELECT  customer_id,
        customer_name,
        LENGTH(TRIM(customer_name)) AS trimmed_length
FROM    commerce_prod.marts.dim_customer
WHERE   TRIM(customer_name) = ''
   OR   customer_name = '';

-- PASS: 0 rows
-- FAIL: log warning; flag for Steward review
```

TC-018 | SCD Type 2 Currency Flag — DQR-018 | High

Field	Value
Rule Reference	DQR-018 (new)
Target Table	dim_customer
DQ Dimension	Validity + Consistency
Tier	1 — High
Rationale	Framework Section 4.3 requires is_current_flag for all dim_ tables. Per SCD Type 2 semantics, exactly one record per natural key must have is_current_flag = TRUE. Multiple current records or no current record both represent data corruption.
Enforcement Tools	dbt: custom SQL test | Snowflake Task

```
-- TC-018a: Multiple current records per customer (SCD violation)
SELECT  customer_id,
        COUNT(*) AS current_record_count
FROM    commerce_prod.marts.dim_customer
WHERE   is_current_flag = TRUE
GROUP BY customer_id
HAVING  COUNT(*) > 1;

-- TC-018b: Customers with NO current record
SELECT  customer_id
FROM    commerce_prod.marts.dim_customer
GROUP BY customer_id
HAVING  SUM(CASE WHEN is_current_flag = TRUE THEN 1 ELSE 0 END) = 0;

-- PASS: 0 rows from both queries
-- FAIL: quarantine affected customer_ids; High alert
```

TC-019 | Staging-to-Curated Row Count Reconciliation — DQR-019 | High

Field	Value
Rule Reference	DQR-019 (new)
Target Table	stg_order → fact_order
DQ Dimension	Consistency + Completeness
Tier	2 — High
Rationale	After deduplication in the staging-to-curated transition, the row count in fact_order must be <= stg_order (deduplication may reduce it). If fact_order has MORE rows than stg_order after a pipeline run, records were inserted outside the controlled pipeline — a data integrity breach.
Enforcement Tools	dbt: custom test | Post-load Snowflake Task

```
-- TC-019: Layer reconciliation check (post-pipeline run)
WITH counts AS (
    SELECT
        (SELECT COUNT(*) FROM commerce_prod.staging.stg_order
         WHERE  audit_batch_id = :current_batch_id)   AS stg_count,
        (SELECT COUNT(*) FROM commerce_prod.marts.fact_order
         WHERE  audit_batch_id = :current_batch_id)   AS fact_count
)
SELECT
    stg_count,
    fact_count,
    fact_count - stg_count                            AS delta,
    CASE
        WHEN fact_count > stg_count THEN 'INTEGRITY_BREACH'
        WHEN fact_count = 0         THEN 'EMPTY_LOAD'
        ELSE 'OK'
    END                                               AS reconciliation_status
FROM counts;

-- PASS: reconciliation_status = 'OK'
-- FAIL: INTEGRITY_BREACH  →  halt pipeline, Critical alert
--       EMPTY_LOAD        →  High alert; investigate ingestion
```

TC-020 | Schema Drift Detection — DQR-020 | Critical

Field	Value
Rule Reference	DQR-020 (new)
Target Table	All Tier 1 & 2 tables
DQ Dimension	Schema Compliance
Tier	1 & 2 — Critical (framework Section 7.3: pipeline pause regardless of tier)

Rationale	Any unplanned column addition, removal, or type change against the declared schema constitutes schema drift. Even additive changes can silently break SELECT * consumers (prohibited by framework Section 6.1 but still present in legacy code) and GE expectations that enumerate columns explicitly.
Enforcement Tools	Snowflake INFORMATION_SCHEMA comparison | dbt compile (detects undeclared columns) | CI/CD PR lineage check

```
-- TC-020: Schema drift detection — compare live schema against declared baseline
-- Baseline declared in dbt schema.yml; snapshot taken at last known-good deploy

WITH declared AS (
    -- Populated from dbt artifacts (manifest.json column list)
    SELECT column_name, data_type
    FROM   governance.schema_baseline
    WHERE  table_name = 'fact_order'
),
live AS (
    SELECT LOWER(column_name) AS column_name,
           LOWER(data_type)   AS data_type
    FROM   information_schema.columns
    WHERE  table_schema = 'MARTS'
      AND  table_name   = 'FACT_ORDER'
)
SELECT
    COALESCE(d.column_name, l.column_name)  AS column_name,
    d.data_type                             AS declared_type,
    l.data_type                             AS live_type,
    CASE
        WHEN d.column_name IS NULL THEN 'UNDECLARED_COLUMN_ADDED'
        WHEN l.column_name IS NULL THEN 'DECLARED_COLUMN_MISSING'
        WHEN d.data_type <> l.data_type THEN 'TYPE_MISMATCH'
        ELSE 'OK'
    END                                     AS drift_status
FROM declared d
FULL OUTER JOIN live l ON d.column_name = l.column_name
WHERE COALESCE(
    CASE WHEN d.column_name IS NULL THEN 1 END,
    CASE WHEN l.column_name IS NULL THEN 1 END,
    CASE WHEN d.data_type <> l.data_type THEN 1 END
) = 1;

-- PASS: 0 rows
-- FAIL: any drift_status != 'OK'  →  Critical alert; pipeline pause
```

5. Cross-Platform Consistency Tests

These tests verify that data processed through both Snowflake and Databricks pipelines produces consistent results. They are the enforcement mechanism for the Consistency dimension of the DQS model. All tests run as daily Snowflake Tasks comparing the two curated layers.

TC-C01 | Record Count Reconciliation

```
-- TC-C01: Snowflake vs. Databricks record count alignment
SELECT
    'snowflake'                          AS platform,
    COUNT(*)                             AS record_count,
    COUNT(DISTINCT order_id)             AS unique_orders
FROM commerce_prod.marts.fact_order
UNION ALL
SELECT
    'databricks',
    COUNT(*),
    COUNT(DISTINCT order_id)
FROM databricks_prod.marts.fact_order;

-- PASS: both record_count values equal; unique_orders equal
-- FAIL: any divergence  →  High alert; consistency incident opened
```

TC-C02 | Financial Aggregate Reconciliation

```
-- TC-C02: Revenue aggregate alignment (tolerance ±0.01%)
WITH platform_totals AS (
    SELECT 'snowflake'  AS platform,
           ROUND(SUM(order_amount), 2) AS total_revenue
    FROM   commerce_prod.marts.fact_order
    UNION ALL
    SELECT 'databricks',
           ROUND(SUM(order_amount), 2)
    FROM   databricks_prod.marts.fact_order
),
comparison AS (
    SELECT
        MAX(CASE WHEN platform = 'snowflake'  THEN total_revenue END) AS
sf_revenue,
        MAX(CASE WHEN platform = 'databricks' THEN total_revenue END) AS db_revenue
    FROM platform_totals
)
SELECT
    sf_revenue,
    db_revenue,
    ABS(sf_revenue - db_revenue)                          AS abs_variance,
    ROUND(ABS(sf_revenue - db_revenue)
          / NULLIF(sf_revenue, 0) * 100, 4)               AS pct_variance,
    CASE
        WHEN ROUND(ABS(sf_revenue - db_revenue)
             / NULLIF(sf_revenue, 0) * 100, 4) > 0.01
        THEN 'RECONCILIATION_FAIL'
        ELSE 'RECONCILED'
    END                                                   AS status
FROM comparison;
```

```
-- PASS: status = 'RECONCILED'  (variance <= 0.01%)
-- FAIL: pipeline hold; Critical alert; finance team notified
```

TC-C03 | Status Distribution Consistency

```
-- TC-C03: Order status distribution parity across platforms
SELECT
    platform,
    order_status_code,
    COUNT(*)                                AS record_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (PARTITION BY platform), 4)
                                            AS pct_distribution
FROM (
    SELECT 'snowflake' AS platform, order_status_code
    FROM   commerce_prod.marts.fact_order
    UNION ALL
    SELECT 'databricks', order_status_code
    FROM   databricks_prod.marts.fact_order
) combined
GROUP BY platform, order_status_code
ORDER BY order_status_code, platform;

-- PASS: pct_distribution matches between platforms for each status
-- FAIL: divergence > 1% for any status  →  High alert
```

6. DQS Composite Test — End-to-End Scoring Validation

The DQS calculation itself is a testable artefact. This test validates that the scoring logic is correct, that weighted components produce the expected composite, and that threshold alerting fires at the correct score boundaries.

```
-- DQS full composite calculation for fact_order
-- Weights: Completeness 30%, Uniqueness 20%, Validity 20%, Timeliness 20%, RI 10%
WITH completeness AS (
    SELECT ROUND(
        ( COUNT(order_id) + COUNT(customer_id)
        + COUNT(order_amount) + COUNT(audit_insertion_timestamp) )
        / NULLIF(COUNT(*) * 4.0, 0) * 100, 4
    ) AS score
    FROM commerce_prod.marts.fact_order
),
uniqueness AS (
    SELECT ROUND(
        (1 - COUNT(d.order_id) * 1.0 / NULLIF(COUNT(*), 0)) * 100, 4
    ) AS score
    FROM commerce_prod.marts.fact_order o
    LEFT JOIN (
        SELECT order_id FROM commerce_prod.marts.fact_order
        GROUP BY order_id HAVING COUNT(*) > 1
    ) d ON o.order_id = d.order_id
),
validity AS (
    SELECT ROUND(
        SUM(CASE WHEN order_amount > 0
                 AND order_status_code IN (
                     'pending','confirmed','shipped','delivered','cancelled')
                 AND order_date <= CURRENT_DATE()
             THEN 1 ELSE 0 END) * 1.0
        / NULLIF(COUNT(*), 0) * 100, 4
    ) AS score
    FROM commerce_prod.marts.fact_order
),
timeliness AS (
    SELECT CASE
        WHEN DATEDIFF('minute',
             MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) <= 60 THEN 100.0
        ELSE 0.0
    END AS score
    FROM commerce_prod.marts.fact_order
),
referential_integrity AS (
    SELECT ROUND(
        (1 - COUNT(CASE WHEN c.customer_id IS NULL THEN 1 END) * 1.0
         / NULLIF(COUNT(*), 0)) * 100, 4
    ) AS score
    FROM commerce_prod.marts.fact_order o
    LEFT JOIN commerce_prod.marts.dim_customer c
           ON o.customer_id = c.customer_id
)
SELECT
    CURRENT_DATE()                                          AS score_date,
    'fact_order'                                            AS dataset,
    'Tier 1'                                                AS tier,
    c.score                                                 AS completeness_score,
```

```
    u.score                                                 AS uniqueness_score,
    v.score                                                 AS validity_score,
    t.score                                                 AS timeliness_score,
    ri.score                                                AS
referential_integrity_score,
    ROUND(
        c.score  * 0.30 +
        u.score  * 0.20 +
        v.score  * 0.20 +
        t.score  * 0.20 +
        ri.score * 0.10, 2
    )                                                       AS dqs_composite,
    CASE
        WHEN ROUND(c.score*0.30 + u.score*0.20 + v.score*0.20
             + t.score*0.20 + ri.score*0.10, 2) >= 95
        THEN 'TIER_1_COMPLIANT'
        WHEN ROUND(c.score*0.30 + u.score*0.20 + v.score*0.20
             + t.score*0.20 + ri.score*0.10, 2) >= 90
        THEN 'MONITOR'
        ELSE 'CRITICAL_BREACH'
    END                                                     AS compliance_status
FROM completeness c, uniqueness u, validity v, timeliness t, referential_integrity
ri;

-- PASS: dqs_composite >= 95  AND compliance_status = 'TIER_1_COMPLIANT'
-- MONITOR: 90 <= dqs_composite < 95  →  Steward review
-- CRITICAL_BREACH: dqs_composite < 90  →  pipeline pause + PagerDuty
```

DQS Score Range	Status	Action	Matches Framework Section
>= 95	**Critical**	TIER_1_COMPLIANT — no action	7.1 Tier 1 threshold
90 – 94	**High**	MONITOR — Steward review within 24h	7.1 Score < 95 warning band
< 90	**Critical**	CRITICAL_BREACH — pipeline pause + PagerDuty + Owner escalation	7.1 Score < 90 trigger
< 75 (Tier2)	**High**	WARNING — Jira ticket auto-created	7.1 Tier 2 threshold
< 60 (Tier3)	**Medium**	INFO — logged only	7.1 Tier 3 threshold

7. Test Execution Plan

Tests are organised by execution phase. Phase 1 runs in DEV against synthetic data to verify rule logic. Phase 2 runs in QA against anonymised production data for integration validation. Phase 3 monitors PROD continuously. This sequence ensures no untested rule ever reaches production monitoring.

Phase	Environment	Test Cases	Trigger	Success Criteria
Phase 1 — Unit Testing	DEV (synthetic data)	TC-001 to TC-020 + C01–C03	Manual / PR trigger	100% of Critical rules pass; all negative injections detected
Phase 2 — Integration Testing	QA (anonymised prod subset)	TC-001 to TC-020 + DQS	Scheduled: post-QA-deploy	DQS >= 95 on QA data; 0 Critical failures
Phase 3 — Regression	QA	Full suite	Every schema change PR	No regressions vs. prior run; all new rules pass
Phase 4 — Production Monitoring	PROD (live data)	TC-009, TC-015, DQS, C01–C03	Continuous / scheduled	No SLA breaches; DQS maintained >= 95 for 30 days
Phase 5 — Quarterly Catalog Review	PROD	Full rules catalog audit	Quarterly (Data Steward)	All rules current; retired rules logged; new risks identified

7.1 Test Entry & Exit Criteria

Entry Criteria

- DEV schema created and seeded with synthetic baseline data (Section 2.2)
- Negative injection scripts tested and cleanup scripts validated
- dbt models compiled and passing dbt compile in DEV environment
- Great Expectations suite configured and checkpoint dry-run successful
- Snowflake Tasks created in DEV warehouse (monitoring_wh equivalent)
- All test cases in this plan reviewed and signed off by Data Steward

Exit Criteria

- All 20 TC test cases executed with documented results (pass/fail per scenario)
- All 3 cross-platform consistency tests passing in QA environment
- DQS composite calculation validated against manual calculation on known dataset
- 0 Critical rule failures outstanding before PROD promotion
- All High failures have Jira tickets open with assigned owner and resolution timeline
- Test results logged to governance.audit_log per framework Section 13.2

8. Full Deliverable Roadmap

This test plan is the third deliverable in a five-document governance suite. Each document has a defined dependency chain — no document should be authored without its predecessors completed and signed off. The roadmap below shows the full sequence, scope, and ownership for the complete case study programme.

#	Deliverable	Status	Depends On	Owner	Primary Output
1	Database Coding & Naming Standards	COMPLETE	—	Data Quality Lead	Authoritative naming conventions, data types, audit columns, prohibited patterns — v2.0 live
2	Case Study — DG & DQ Transformation	COMPLETE	Doc 1	Data Quality Lead	End-to-end narrative: baseline → root cause → governance → implementation → results
3	Data Quality Test Plan ◀ this document	IN PROGRESS	Doc 1 + Doc 2	Data Quality Lead	20 test cases (DQR-001–020), cross-platform tests, DQS validation, data strategy, execution plan
4	Data Quality Rules Catalog	NEXT	Doc 3	Data Steward + DQ Lead	Full registry of DQR-001–020 + extension rules, with activation status, owner, severity, review cadence
5	Data Quality Monitoring & DQS Dashboard	PLANNED	Doc 3 + Doc 4	Data Custodian + DQ Lead	Live DQS dashboard, Snowflake Task schedule, alerting runbooks, incident response playbooks

8.1 Document 4 Preview — Data Quality Rules Catalog

The Rules Catalog will be the operational registry for all DQ rules across the platform. It extends the existing DQR-001 to DQR-010 catalog in the Governance Framework with the ten new rules defined in this test plan, plus a structured intake process for future rules.

Rules Catalog Structure (Doc 4)
▸ Full rule registry: DQR-001 to DQR-020 + intake template for new rules
▸ Per-rule: target table, validation SQL, severity, tool mapping, activation date, review date

- Rule lifecycle states: Draft → Steward Review → Active → Deprecated

- Cross-reference to test case (TC-XXX) for every active rule
- Exception registry: rules with approved exceptions, expiry dates, compensating controls
- Ownership matrix: Data Steward assigned per rule; escalation path documented

8.2 Document 5 Preview — Monitoring & DQS Dashboard

The Monitoring document will translate this test plan into production operational infrastructure: the Snowflake Task and Alert schedule, the DQS dashboard specification, incident response playbooks per severity level, and the integration between the DQS pipeline and the enterprise data health dashboard.

Monitoring & DQS Scope (Doc 5)

- Snowflake Task schedule: full task tree with dependencies, warehouses, and cron expressions

- DQS pipeline: nightly calculation → results table → dashboard feed → alerting

- Slack alerting playbook: which alerts fire to which channel for each rule and severity

- PagerDuty integration: Critical rule failures → on-call rotation → escalation ladder

- BI dashboard specification: DQS by dataset, trend over time, rule failure heatmap
- Incident response playbook: per-severity runbook with investigation steps and resolution SLAs
- Quarterly review checklist: rules catalog audit, tier re-assessment, DQS trend analysis

9. Test Results Log Template

All test execution results must be recorded in this format and logged to governance.audit_log (framework Section 13.2). The log is the audit trail for DQ programme compliance and is referenced in quarterly governance reviews.

Test Case	Run Date	Environment	Rows Tested	Violations Found	Result	Action Taken	Ticket Ref
TC-001	YYYY-MM-DD	DEV / QA / PROD	n	n	**PASS**	None required	—
TC-001	YYYY-MM-DD	DEV / QA / PROD	n	n	**FAIL**	Pipeline halted; P1 opened	JIRA-XXX
TC-002	—	—	—	—	**PENDING**	—	—

Completed logs are stored in the test-results/ directory of the governance repository, one file per test run date, version-controlled. Failed test logs are referenced in the Jira ticket created at failure time and linked in the governance.audit_log entry.

Test Plan Sign-Off Requirements
▸ Data Quality Lead — Test plan authored and all test cases reviewed
▸ Data Steward — DQ rules validated; severity classifications confirmed ▸ Data Owner (Tier 1 datasets) — SLA thresholds and escalation paths approved
▸ Data Custodian — Test environment setup confirmed; injection scripts validated
▸ Before promotion to PROD monitoring: 0 Critical failures, all High failures ticketed